

Nanoparticle Toxicity Prediction Using Data-Driven Methods

Step-by-Step Project Procedure Report

Team Member	Enrolment No.
Pratham Joshi	KU2507U0361
Divyansh Khatri	KU2507U0188
Dhyey Bhatt	KU2507U0138
Jinay Shah	KU2507U0070

Final model: Random Forest | Accuracy 94.4% | F1 0.865

1. Aim of the Project

Nanoparticles are widely used in drug delivery because they are extremely small, have a very large surface area for their size, and can be directed towards specific cells and tissues. The problem is that these very same properties can also make them toxic to healthy cells. Testing every newly designed particle in a laboratory is slow and expensive, so it is useful to be able to screen a particle computationally before it ever reaches the lab bench.

The aim of this project is therefore to build a working machine learning system that takes the measurable physicochemical properties of a nanoparticle and classifies it as **TOXIC** or **NON-TOXIC**.

Objectives

1. Collect a real, published nanoparticle toxicity dataset.
2. Understand, clean and preprocess the data.
3. Explore the data and identify which properties influence toxicity.
4. Train and compare five machine learning classifiers.
5. Evaluate them properly and select the best one.
6. Build a prediction system that anybody can use.
7. Test the whole thing and fix whatever is broken.

2. Software and Libraries Used

Component	What we used	Why
Language	Python 3.12	Standard for data science work
Data handling	pandas, numpy	Reading, cleaning and reshaping the dataset
Machine learning	scikit-learn	All five classifiers, scaling, metrics, cross validation
Plots	matplotlib, seaborn	Histograms, box plots, heatmap, ROC curves
Saving the model	joblib	Storing the trained pipeline to a file
Web app	Streamlit	Front end for the final prediction system
Testing	pytest	The bug sweep at the end

3. Step-by-Step Procedure

We followed the fifteen step research workflow. Each step below explains what we did, how we did it, and what came out of it.

1 Define the Research Problem

We first fixed exactly what the model should answer. We settled on a **binary classification problem**: given the properties of a nanoparticle and the exposure conditions, predict whether it will be toxic to cells or not. We deliberately did not attempt to predict the exact cell viability percentage, because a yes/no toxicity flag is what is actually useful for early screening, and it is a much more achievable target with a small dataset.

2 Literature Review

We read around the topic of nanotoxicity and machine learning based prediction (nano-QSAR). The main things we took away were:

- Toxicity does not depend on any single property. Size, charge, dose and exposure time act together, so a multivariate model makes sense.
- Published nanotoxicity datasets are small and heavily imbalanced, so accuracy on its own is a misleading metric.
- The chemistry of the material itself (which metal oxide it is) is often a stronger signal than the particle geometry.

This review is also how we found our dataset, from the NanoTox study.

3 Dataset Collection

We used the **NanoTox metal-oxide nanoparticle dataset**, which is published openly alongside a peer reviewed study.

Item	Detail
Source repository	github.com/NanoTox/ToxicityModel (file <code>dataset.txt</code>)
Reference	<i>NanoTox: Development of a parsimonious in silico model for toxicity assessment of metal-oxide nanoparticles using physicochemical features</i> , bioRxiv 2021, doi:10.1101/2021.02.22.432301
Records	483 experiments
Columns	24
Materials	Al ₂ O ₃ , CuO, Fe ₂ O ₃ , TiO ₂ , ZnO
Cell lines	11 different cell lines
Format	Tab separated text

Rather than downloading it by hand we wrote `src/download_data.py` so that the whole project can be reproduced from an empty folder:

```
python src/download_data.py
```

```
Downloading from https://raw.githubusercontent.com/NanoTox/ToxicityModel/master/dataset.txt ...  
Saved nanotox_dataset.tsv (58.4 KB)
```

4 Data Understanding

Before changing anything we simply looked at the data and printed a report of what we had. This is done by `describe_data()` in `src/preprocess.py`.

```
Rows: 483   Columns: 24  
Missing values per column:  
  No missing values found.  
Duplicate rows: 6  
Target variable distribution:  
  nonToxic   405 (83.9%)  
  Toxic       78 (16.1%)  
-> The dataset is imbalanced, so accuracy alone is not enough.
```

What we learned at this stage:

- There are no missing values, which is unusually clean.
- The classes are badly imbalanced, roughly 84% to 16%.
- There are a handful of exactly duplicated rows.
- There is a `viability` column sitting right next to the `class` column, which we flagged as suspicious.

Important observation. The `class` label (Toxic / nonToxic) was created by the dataset authors *from* the `viability` value. So `viability` is not an input property, it is the answer in disguise. If we had used it as a feature every model would have scored close to 100% and the project would have proved nothing. We excluded it from the very beginning.

5 Data Preprocessing

This is where the data was actually cleaned. The steps, in order:

1. **Remove duplicate rows** – 6 exact duplicates removed.
2. **Encode the target** – `Toxic` becomes 1, `nonToxic` becomes 0.
3. **Drop the leaking column** – `viability` is removed and listed in `config.LEAKY_COLS`.
4. **Force numeric types** and fill any missing numeric value with the column median. There were none here, but the step stays so the pipeline does not break on a different dataset.
5. **Merge repeated experiments** – explained below.
6. **Train/test split** – 75% train, 25% test, stratified so both halves keep the same toxic ratio.
7. **Feature scaling** – `StandardScaler`, but placed *inside* a scikit-learn `Pipeline` so it is only ever fitted on the training fold.

The problem we nearly missed. Once `viability` is removed, a lot of rows become *identical*. The raw file records the same experiment (same particle, same dose, same cell line) several times, each with a slightly different measured viability. With those repeats present, the random train/test split was putting copies of the same experiment into both the training set and the test set, which meant our test scores were not honest.

We fixed this with `collapse_replicates()`, which keeps one row per unique experiment. Where the repeats disagreed on the label we took the majority, and an exact tie counts as toxic, because for a safety tool a false warning is safer than a false all-clear.

```
Removed 6 duplicate rows.
Encoded target: Toxic = 1, nonToxic = 0
Dropped leaky column(s): ['viability']
Merged 122 repeated experiments (477 rows -> 355 unique experiments).
  11 of them had repeats that disagreed on the label - majority vote used.
Final clean shape: (355, 24)
```

Stage	Rows
Raw dataset as downloaded	483
After removing exact duplicates	477
After merging repeated experiments	355
Toxic samples in the final data	68 (19.2%)

How the columns were divided

Thirteen of the descriptors are pure chemistry and are *the same for every row of a material* (Al_2O_3 always has the same molecular weight). We separated these from the six properties a user can actually measure, so that the final app only has to ask for six numbers and can look up the rest itself.

Group	Columns
Entered by the user (6)	<code>coresize</code> , <code>hydrosized</code> , <code>surfcharge</code> , <code>surfarea</code> , <code>Exptime</code> , <code>dosage</code>
Looked up from the material (13)	<code>Hsf</code> , <code>Ec</code> , <code>Ev</code> , <code>MeO</code> , <code>enthalpy</code> , <code>ratio</code> , <code>e</code> , <code>esum</code> , <code>esumbyo</code> , <code>MW</code> , <code>NMetal</code> , <code>NOxygen</code> , <code>ox</code>
Categorical (1)	<code>Celltype</code> (Cancer / Normal)
Target	<code>class</code> → <code>toxic</code> (1 / 0)
Never used	<code>viability</code> (leaks the answer)

The material lookup table that was generated

NPs	Hsf	Ec	Ev	MeO	enthalpy	ratio	e	esum	esumbyo	MW	NMetal	NOxygen	ox
Al ₂ O ₃	-17.345	-1.51	-9.81	5.67	1187.83	5.556	1.61	3.22	1.073	101.960	2	3	3
CuO	-1.609	-5.17	-6.51	5.87	713.74	2.740	1.90	1.90	1.900	79.546	1	1	2
Fe ₂ O ₃	-8.512	-4.99	-6.99	5.98	1363.40	5.455	1.83	3.66	1.220	159.600	2	3	3
TiO ₂	-9.779	-4.16	-7.49	5.77	1575.73	6.557	1.54	1.54	0.770	79.860	1	2	4
ZnO	-3.608	-3.89	-7.20	5.67	662.44	2.703	1.65	1.65	1.650	81.380	1	1	2

6 Exploratory Data Analysis

We then plotted the data to see what it actually looks like. All of this is in `src/eda.py` and the figures are saved to `results/figures/`.

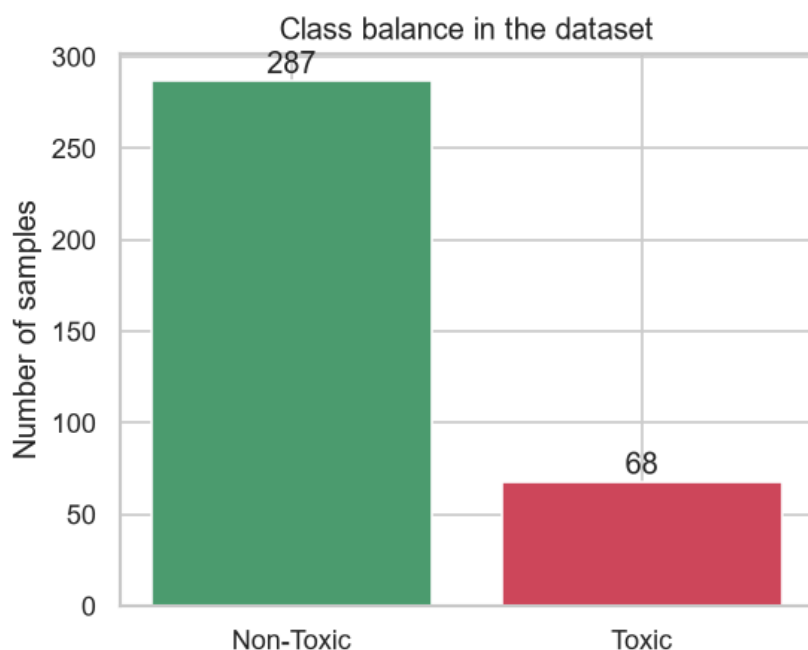


Figure 1 – Class balance. The dataset is clearly imbalanced.

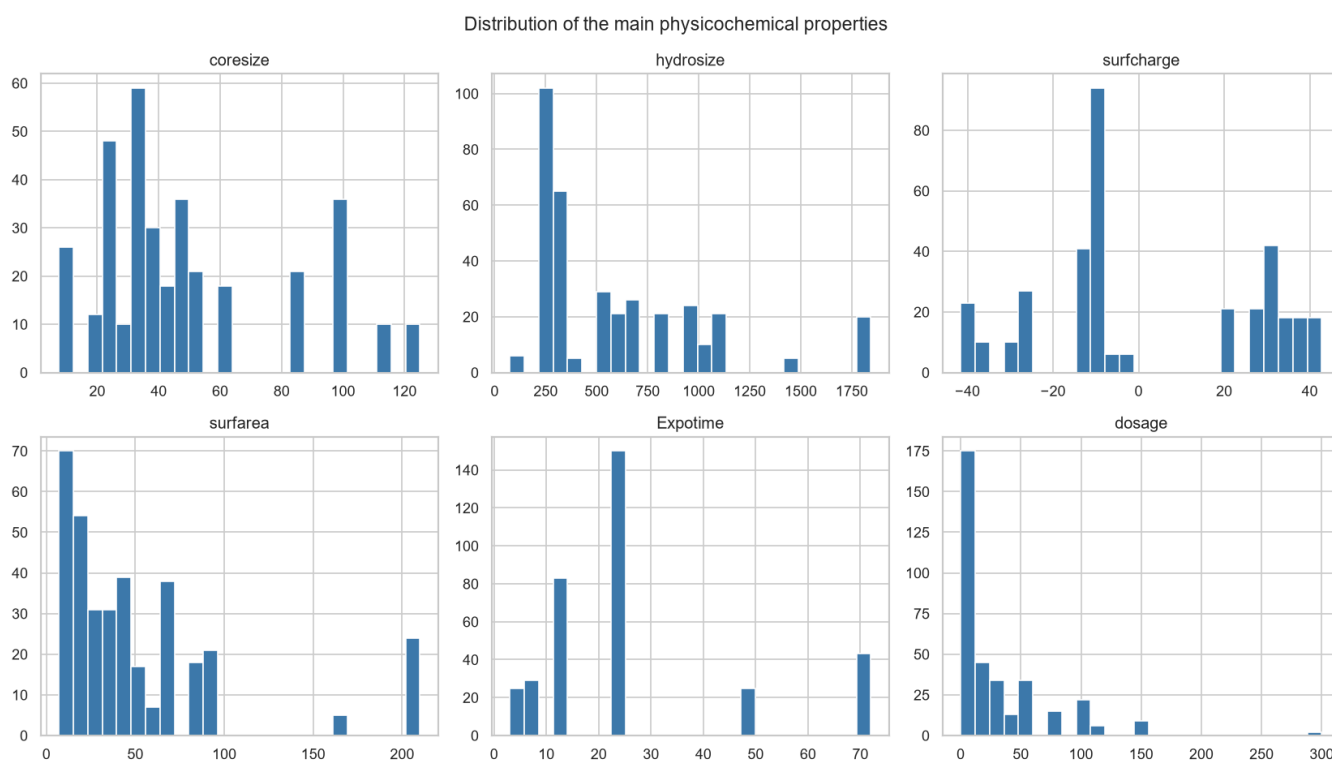


Figure 2 – Distribution of the six measurable properties.

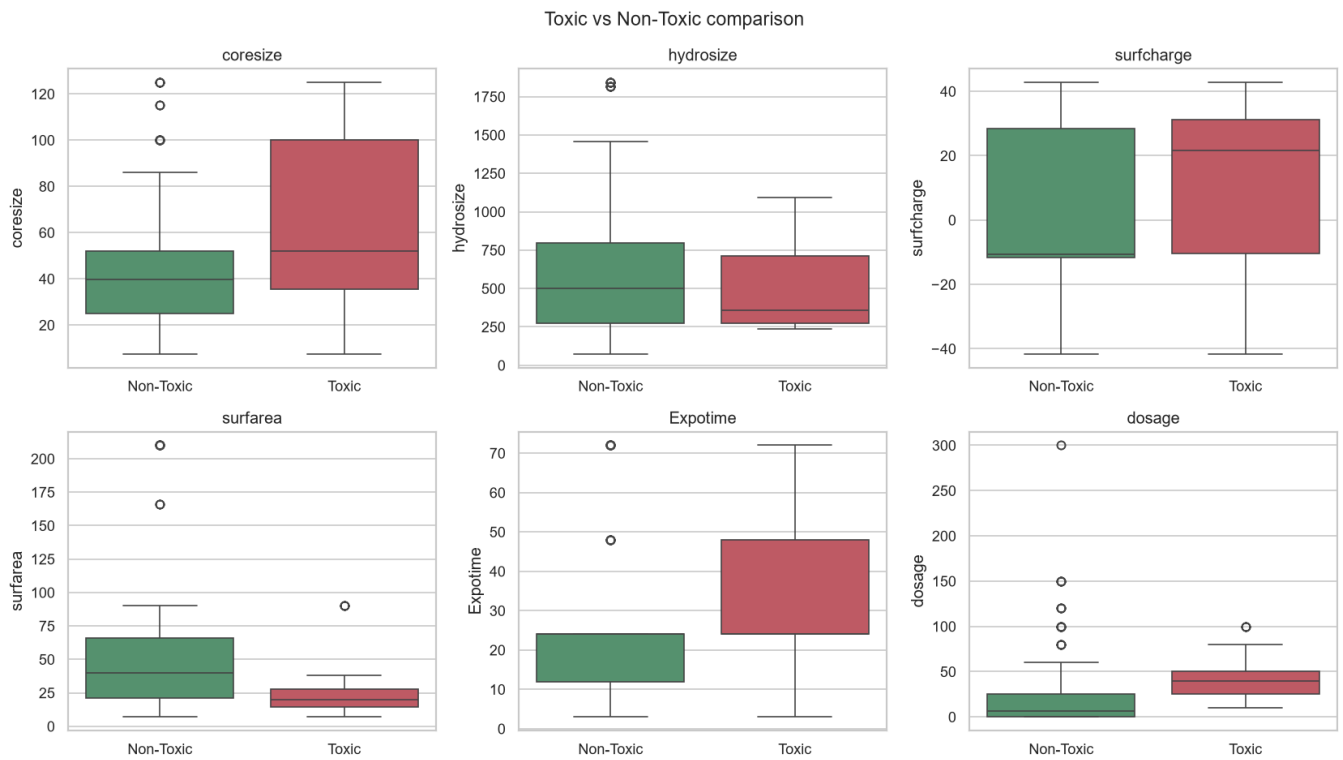


Figure 3 – Toxic versus non-toxic groups compared property by property.

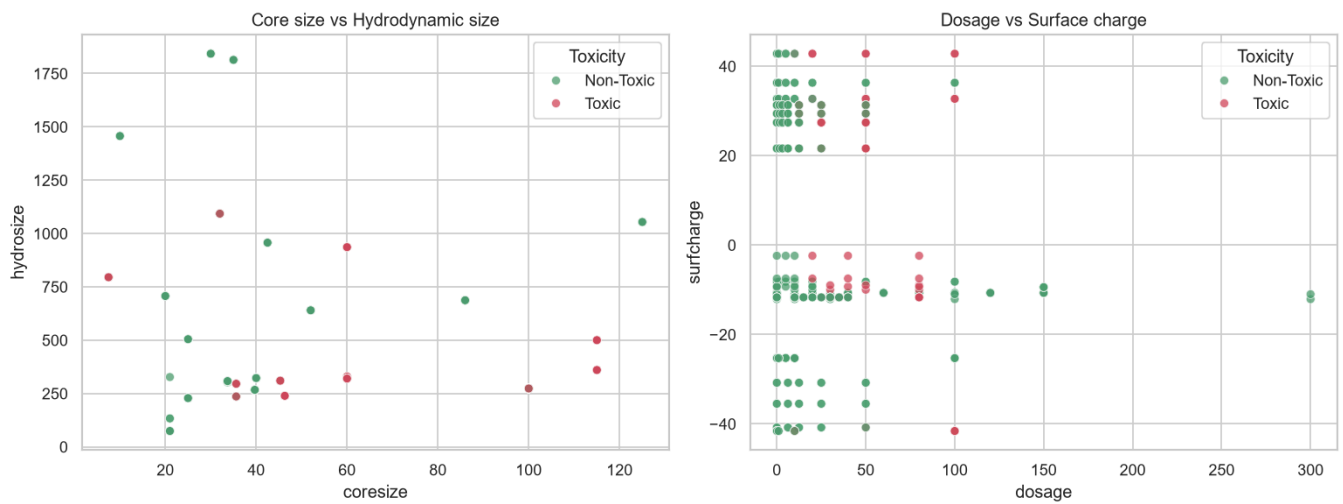


Figure 4 – Scatter plots showing how the two classes separate.

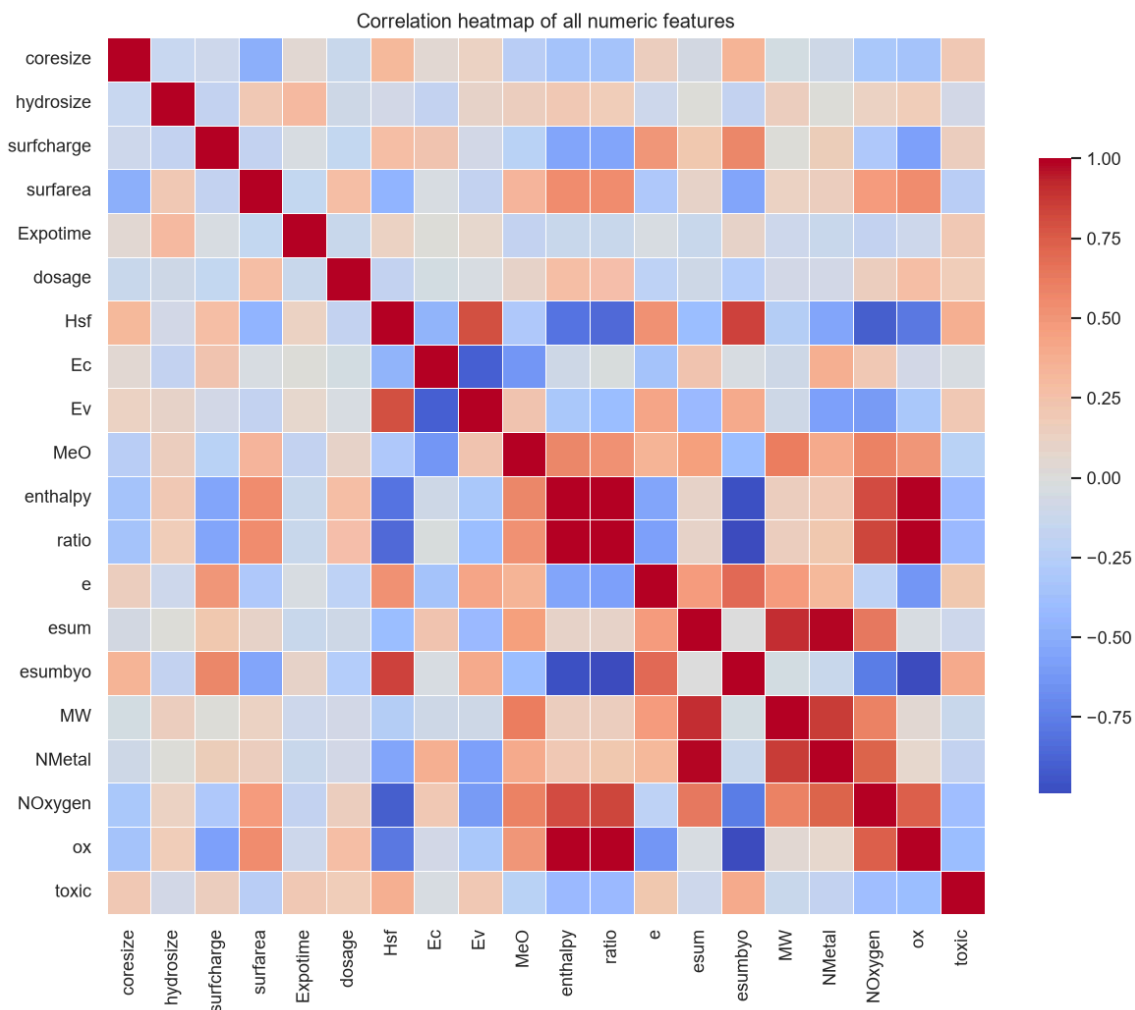


Figure 5 – Correlation heatmap of all numeric features.

7 Feature Analysis

From the correlation analysis, these are the features most strongly related to toxicity:

Feature	Correlation with toxicity
ratio	-0.415
enthalpy	-0.410
esumbyo	0.406
ox	-0.400
NOxygen	-0.386
Hsf	0.383
surfarea	-0.241
MeO	-0.227

The striking result is that the strongest signals are all **material-level chemistry descriptors** – the metal to oxygen ratio, the enthalpy, the oxidation state – rather than the particle geometry. In plain language, *what the particle is made of matters more than how big it is*.

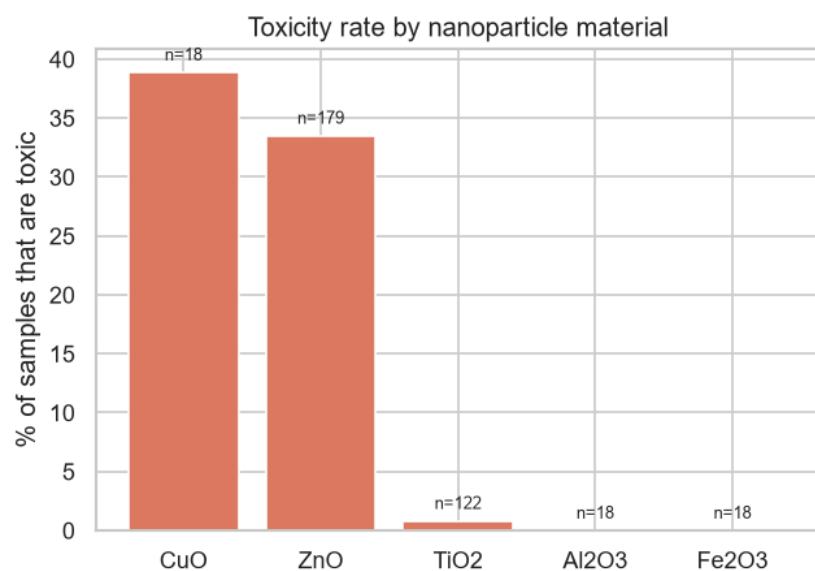


Figure 6 – Toxicity rate by material.

Material	Samples	Toxic	% toxic
CuO	18	7	38.9
ZnO	179	60	33.5
TiO2	122	1	0.8
Al2O3	18	0	0.0
Fe2O3	18	0	0.0

CuO and ZnO account for essentially all of the toxicity in this dataset, which matches the chemistry – both dissolve and release metal ions inside cells. Al₂O₃ and Fe₂O₃ are inert here.

8 Model Development

We built the five classifiers named in our abstract. Every one of them is wrapped in a scikit-learn `Pipeline` together with the scaler and the one-hot encoder, so that no information from the test set can leak into the scaling.

#	Model	Settings we chose
1	Logistic Regression	<code>max_iter=2000</code> , <code>class_weight="balanced"</code>
2	Decision Tree	<code>max_depth=6</code> , <code>class_weight="balanced"</code>
3	K-Nearest Neighbors	<code>n_neighbors=5</code>
4	Support Vector Machine	RBF kernel, <code>probability=True</code> , <code>class_weight="balanced"</code>
5	Random Forest	<code>n_estimators=300</code> , <code>class_weight="balanced"</code>

Why `class_weight="balanced"` ? Only 19% of the samples are toxic. Without this setting the models simply learn to answer "non-toxic" to everything, which still scores about 81% accuracy while being completely useless. Balancing the class weights forces the models to actually pay attention to the toxic minority.

9 Model Training

Each pipeline was fitted on the training portion (266 experiments). On top of the single train/test split we also ran **5-fold stratified cross validation** on the training data, so that we could see how stable each model is rather than trusting one lucky split.

Train: 266 rows Test: 89 rows
Toxic in train: 51 Toxic in test: 17

10 Model Testing

Each trained model then predicted on the 89 unseen test experiments, producing both a class label and a probability of toxicity.

11 Performance Evaluation

We recorded accuracy, precision, recall, F1 score and ROC-AUC, plus a confusion matrix for every model.

Model	Accuracy	Precision	Recall	F1	ROC_AUC	CV_F1_mean	CV_F1_std
Random Forest	0.944	0.800	0.941	0.865	0.946	0.880	0.057
Decision Tree	0.921	0.778	0.824	0.800	0.904	0.831	0.057
Logistic Regression	0.888	0.667	0.824	0.737	0.931	0.695	0.098
SVM	0.865	0.609	0.824	0.700	0.928	0.665	0.078
KNN	0.876	0.875	0.412	0.560	0.836	0.527	0.114

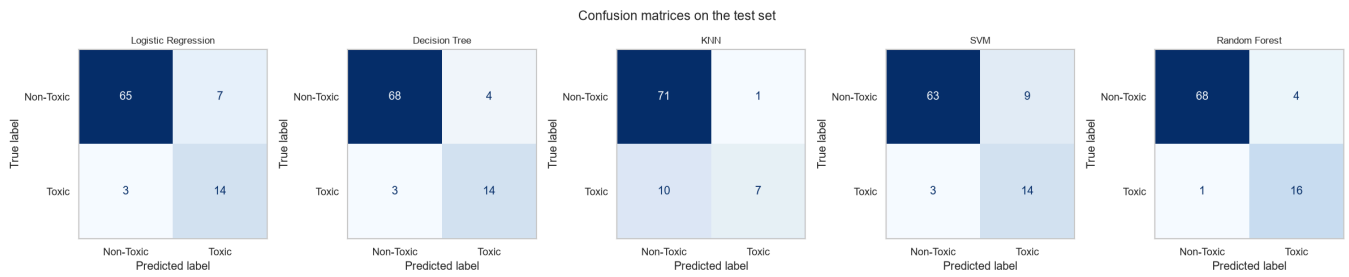


Figure 7 – Confusion matrices on the test set.

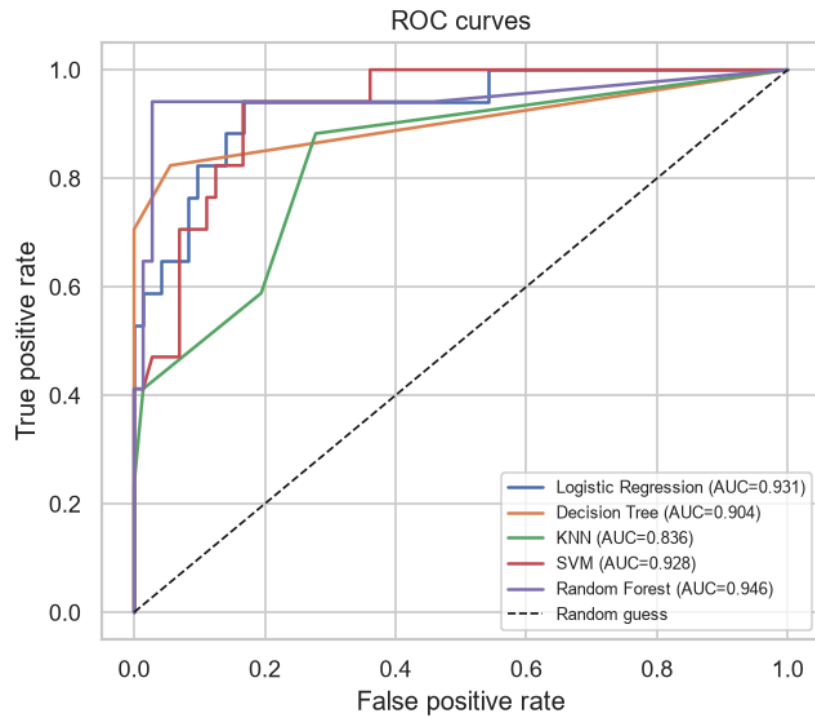


Figure 8 – ROC curves for all five models.

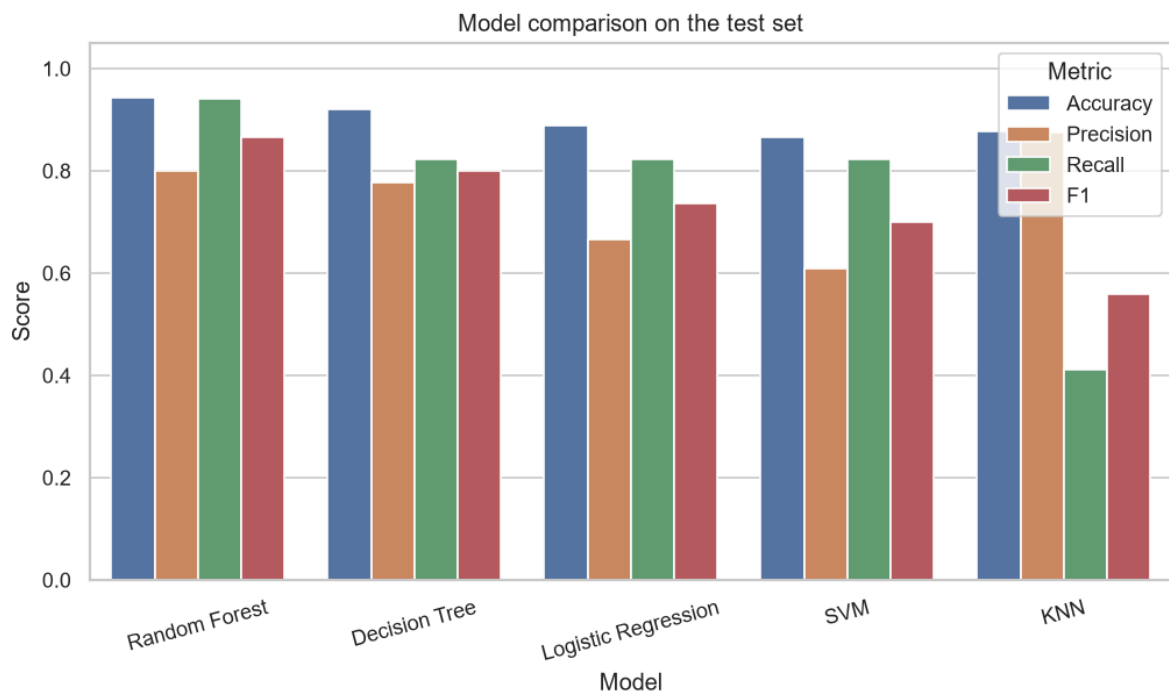


Figure 9 – Model comparison across all metrics.

12 Selecting the Best Model

We selected the best model by **F1 score, not accuracy**. The reason is visible in the results table: KNN reaches 87.6% accuracy, which looks respectable, but its recall is only 0.412 – it misses almost 60% of the toxic particles. For a safety screening tool, failing to flag a toxic particle is the worst possible mistake, so a metric that balances precision and recall is the right one to optimise.

Final model: Random Forest

Accuracy 0.944 · Precision 0.800 · Recall 0.941 · F1 0.865 · ROC-AUC 0.946

It correctly identifies 94% of the toxic nanoparticles in the test set. The winner is then retrained on all 355 experiments and saved to `models/best_model.joblib`.

13 Interpretation and Insights

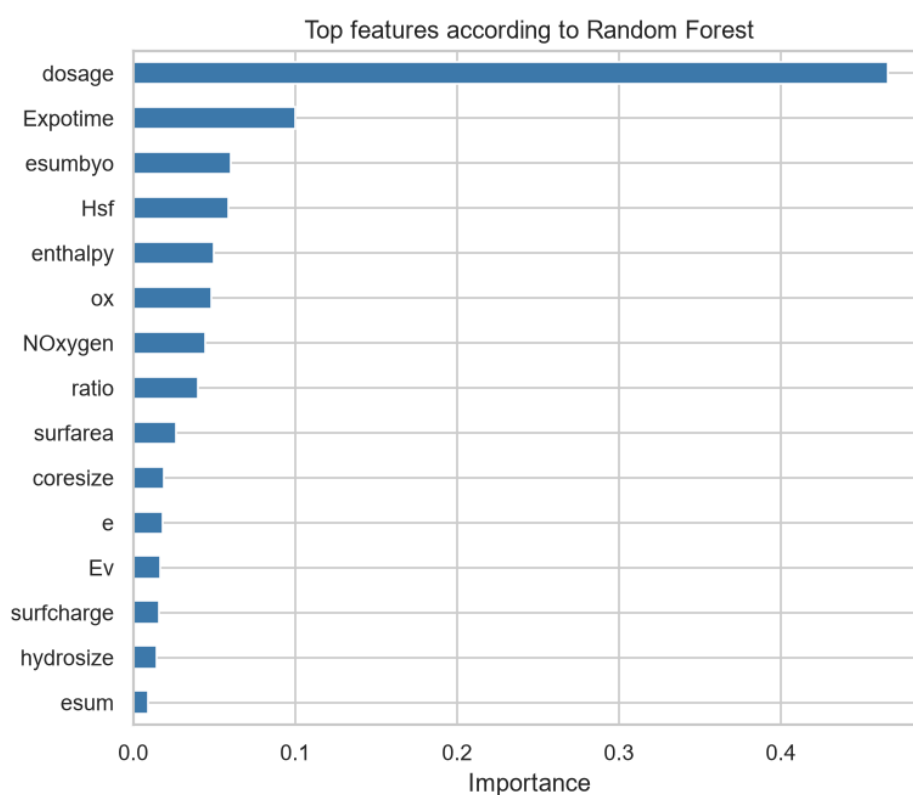


Figure 10 – Feature importance from the Random Forest.

Feature	Importance
dosage	0.4660
Exptime	0.0999
esumbyo	0.0601
Hsf	0.0588
enthalpy	0.0496
ox	0.0483
NOxygen	0.0445
ratio	0.0398
surfarea	0.0268
coresize	0.0190

What the model is telling us:

- The chemical identity of the material dominates. The descriptors that encode *which* oxide it is are the most informative features.
- Among the properties an experimenter can actually control, **dosage**, **exposure time** and **surface area** carry the most weight.
- Hydrodynamic size turned out to be almost uninformative here (correlation with toxicity of only -0.02).
- Practical takeaway: choosing a safer base material matters more than fine-tuning the particle size.

14 Final Prediction System

We built two front ends, both of which call the exact same prediction function so they can never disagree with each other.

a) Web application (Streamlit)

```
streamlit run app.py
```

The user picks a material and types in the six measurable properties. The app fills in the thirteen chemistry descriptors automatically from the material lookup table, runs the saved model, and displays TOXIC or NON-TOXIC together with a probability. It has three tabs: the prediction form, the model results with all the figures, and an about page.

b) Command line tool

```
python src/predict.py --material ZnO --coresize 40 --hydrosized 300 \
  --surfcharge -12 --surfarea 50 --expotime 24 --dosage 100
```

```
=====
      NANOPARTICLE TOXICITY PREDICTION
=====
Material          : ZnO
Core size         : 40.0 nm
Hydrodynamic size : 300.0 nm
Surface charge    : -12.0 mV
Surface area      : 50.0 m2/g
Exposure time     : 24.0 hours
Dosage            : 100.0 ug/mL
Cell type         : Cancer
-----
PREDICTION        : TOXIC
Probability toxic  : 99.0%
Model used        : Random Forest
=====
```

It also runs in an interactive question-by-question mode if no arguments are given, and has a `--demo` flag that runs four worked examples.

Sample outputs from `--demo`

Material	Dose	Cell type	Prediction	Probability toxic
ZnO	100 µg/mL	Cancer	TOXIC	99.0%
CuO	200 µg/mL	Cancer	TOXIC	96.7%
TiO ₂	10 µg/mL	Normal	NON-TOXIC	1.3%
Al ₂ O ₃	0.01 µg/mL	Normal	NON-TOXIC	0.0%

These agree with the known chemistry: CuO and ZnO release toxic metal ions, while TiO₂ and Al₂O₃ are largely inert.

15 Bug Sweep, Conclusion and Future Work

Once everything worked we deliberately went back and tried to break it. We wrote a test suite of 18 tests in `tests/test_pipeline.py` and found seven issues. The full write-up is in `notes/BUG_SWEEP.md`; the summary is in section 5 of this report.

```
pytest -v
...
18 passed
```

4. Results

Model	Accuracy	Precision	Recall	F1	ROC_AUC	CV_F1_mean	CV_F1_std
Random Forest	0.944	0.800	0.941	0.865	0.946	0.880	0.057
Decision Tree	0.921	0.778	0.824	0.800	0.904	0.831	0.057
Logistic Regression	0.888	0.667	0.824	0.737	0.931	0.695	0.098
SVM	0.865	0.609	0.824	0.700	0.928	0.665	0.078
KNN	0.876	0.875	0.412	0.560	0.836	0.527	0.114

Random Forest is the final model, with 94.4% accuracy, an F1 score of 0.865 and a ROC-AUC of 0.946 on the held-out test set.

Honest limitation: leave-one-material-out

Almost all the toxic samples in this dataset are CuO or ZnO, and thirteen of the nineteen numeric features are constant within a material. That means a normal random split allows the model to partly *recognise the material* rather than learn the underlying chemistry, and the reported accuracy would be optimistic for a genuinely new nanoparticle.

To measure how serious this is, we ran an extra test that was not in the original plan: train on four materials and test on the fifth, which the model has never seen at all.

Held_out_material	n_samples	n_toxic	Accuracy	Recall_toxic
Al2O3	18	0	1.000	–
CuO	18	7	0.889	0.714
Fe2O3	18	0	1.000	–
TiO2	122	1	0.992	0.000
ZnO	179	60	0.771	0.717

Held-out ZnO accuracy drops to 0.771, against 0.944 on the normal test split. The model should therefore be used for materials similar to the five it was trained on, and **not** trusted for a completely new nanoparticle. This is a limitation of the dataset (only 5 materials), not of the code.

5. Bug Sweep Summary

#	Issue	Severity	Fix
1	The <code>viability</code> column leaked the answer	Critical	Dropped in preprocessing; a test fails if it returns
2	Repeated experiments split across train and test	High	<code>collapse_replicates()</code> ; 477 → 355 unique experiments
3	The web app had its own copy of the prediction code	Medium	App now imports the shared <code>predict()</code> function
4	Column order was never verified	Medium	Feature order saved with the model and reapplied every time
5	Accuracy was hiding a model with 0.41 recall	Medium	Best model selected by F1; balanced class weights
6	No validation on user input	Low	Clear <code>ValueError</code> for unknown material or cell type
7	No warning when input is outside the training range	Low	<code>check_ranges()</code> warns in both the app and the CLI

Two of these actually changed our results. Bug 2 was found by a test we expected to pass, and fixing it **changed which model won** – the Decision Tree had been coming first partly because it could memorise the duplicated rows. After the fix, Random Forest wins and the held-out ZnO recall improved from 0.257 to 0.717.

6. Conclusion

We built a complete, working, end-to-end system. It downloads a real published dataset, cleans it, explores it, trains and compares five classifiers, evaluates them properly, selects the best one, and serves predictions through both a web app and a command line tool. The final Random Forest model reaches 94.4% accuracy and correctly flags 94% of the toxic nanoparticles in the test set.

Just as importantly, we tested our own work and documented what was wrong with it. Two of the seven bugs we found would have made our results look considerably better than they really are, and we would rather report an honest 94% than a fake 100%.

Future work

- Obtain a larger dataset with more than five materials, so the leave-one-material-out scores improve.
- Extend beyond metal oxides to polymeric and silica nanoparticles.
- Try SMOTE or other resampling techniques for the class imbalance instead of relying only on `class_weight="balanced"`.
- Use SHAP values so that each individual prediction can be explained, not just the overall feature importances.
- Tune hyperparameters systematically with `GridSearchCV`.

7. How to Reproduce Everything

```
pip install -r requirements.txt

python run_all.py          # download -> clean -> EDA -> train
streamlit run app.py       # launch the web app
pytest -v                  # run the 18 tests
```

Deleting the `data/`, `models/` and `results/` folders and running `python run_all.py` rebuilds the entire project from scratch, including re-downloading the dataset. Every random seed is fixed at 42, so the numbers in this report come out identical every time.

8. Files Submitted

File	What it is
<code>dataset/1_raw_nanotox_dataset.csv</code>	The original dataset as downloaded (483 rows)
<code>dataset/2_cleaned_dataset_used_for_training.csv</code>	The cleaned data the model was actually trained on (355 rows)
<code>dataset/3_material_properties_lookup.csv</code>	The chemistry descriptors per material
<code>src/config.py</code>	All paths and column definitions
<code>src/download_data.py</code>	Step 3
<code>src/preprocess.py</code>	Steps 4 and 5
<code>src/eda.py</code>	Steps 6 and 7
<code>src/train_models.py</code>	Steps 8 to 12
<code>src/predict.py</code>	Step 14, command line
<code>app.py</code>	Step 14, web app
<code>tests/test_pipeline.py</code>	Step 15, the 18 tests
<code>notes/BUG_SWEEP.md</code>	Full bug sweep write-up
<code>results/</code>	All metrics tables and the ten figures
<code>models/best_model.joblib</code>	The trained Random Forest pipeline